



Agentic AI

DSPy: Programming—Not Prompting—Language Models



DSPy 3.x (3.1.3) · `pip install dspy`

What You'll Build, Measure, and Think



Typed Pipelines

Write Signatures with typed I/O fields and chain them into composable Modules. No more unstructured prompt strings.



Auto-Optimized

Run BootstrapFewShot and MIPROv2 to automatically tune any pipeline from labeled examples. Understand when optimization helps—and when it cannot.



Constrained Output

Add assertions to enforce output correctness with automatic self-correction retries. Learn to design metrics before you design pipelines.



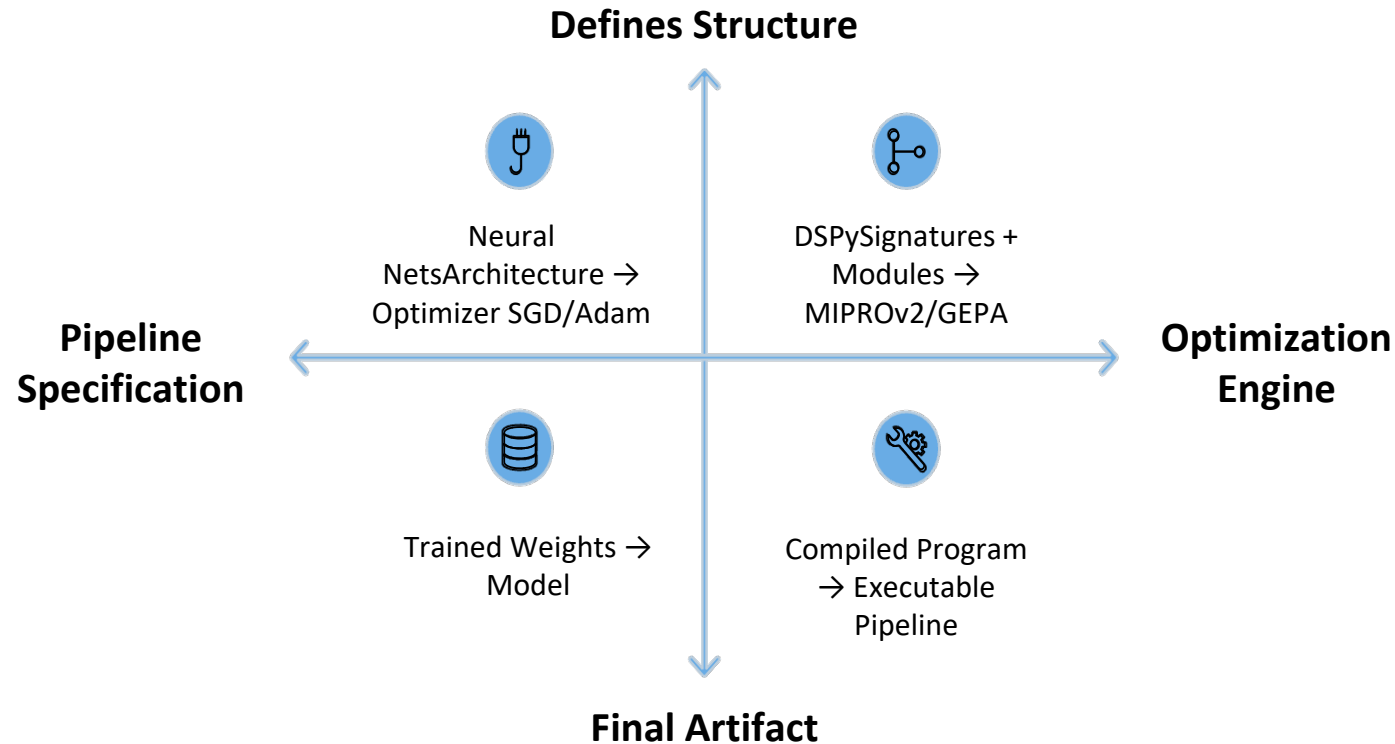
Durable Thinking

Understand the compiler analogy behind DSPy so you can evaluate any future LM optimization framework on first principles.

01

The Mental Model: DSPy as a Compiler

The Compiler Analogy: Why It Changes Everything



Stop writing prompts, start writing programs.

The prompt is a parameter, not code.

"Better prompts" is a measurement question.

You run experiments, not vibes.
Optimization is empirical.

Model swaps become free.

The optimizer re-derives prompts for each backend from the same source program.

In deep learning, you define the architecture — not the individual weight values. The optimizer finds the weights from data. In DSPy, you define the pipeline — not the prompt strings. The optimizer finds the prompts from examples and a metric.

- ❑ This mental model persists beyond DSPy. Any system that separates pipeline structure from prompt parameters is using the same idea.

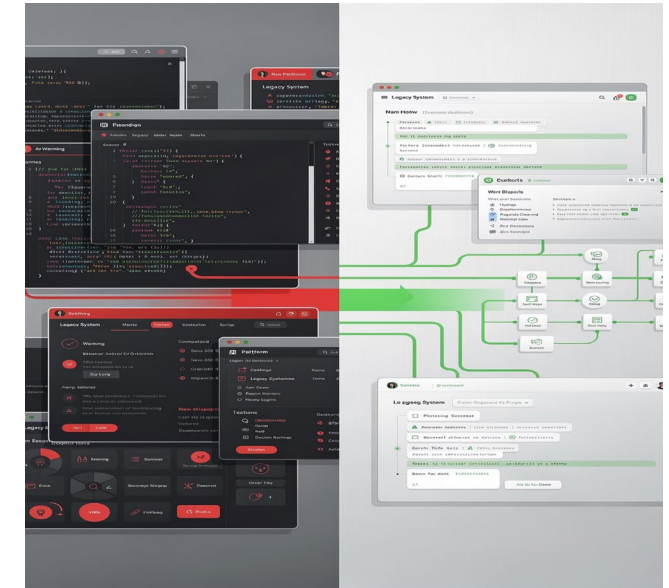
Prompt Engineering vs. DSPy: The Fundamental Shift

✗ Before—Prompt Engineering

- Raw strings in .txt files — no type system, no tests, no compiler
- Breaks on every model upgrade: GPT-3.5 → GPT-4 prompts rarely transfer
- Teams maintain 40+ versions: `prompt_v3_final_FINAL.txt` is a real cliché
- Impossible to audit, diff, or version-control reliably

✓ After—DSPy

- Typed Signatures — a Python class, not a string
- Model-agnostic: swap LLM backends in one line; the optimizer re-tunes automatically
- Compiled to JSON — git-diffable, reproducible, shareable across environments
- Optimizes itself automatically from your labeled examples and metric



Why DSPy Wins in Its Wheelhouse

Only Framework With Automatic Prompt Optimization From Data

No other mainstream tool does what MIPROv2 does—generate, evaluate, and select prompt instructions without human iteration.

Model-Agnostic by Design

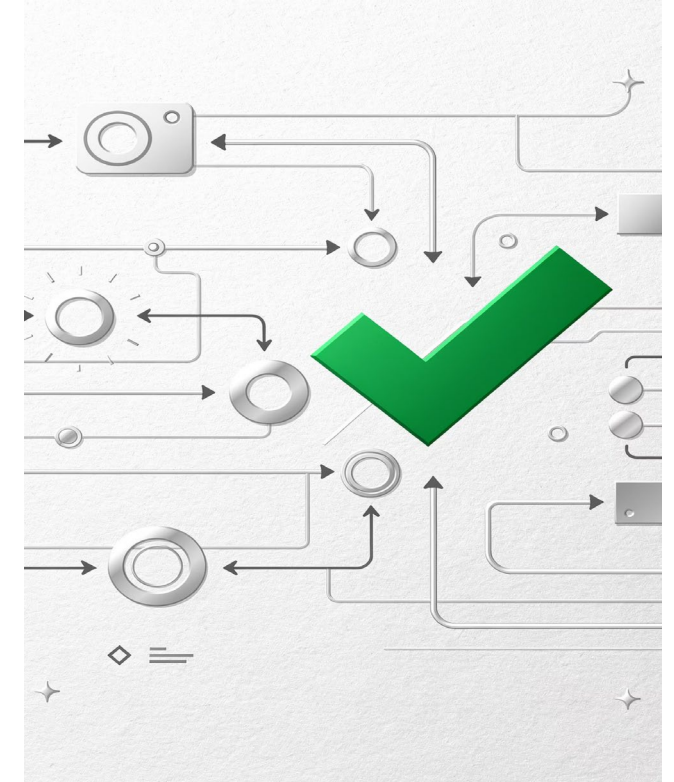
The same Signature runs on GPT-4o, Claude 3.5, or local Llama. One codebase, any backend, re-optimized automatically.

Compiled Programs Are Auditable Artifacts

JSON-serializable, git-diffable, and reproducible—critical in regulated industries where "what prompt was used?" must have a deterministic answer.

Typed Signatures Catch Structural Errors at Definition Time

`Literal["high", "medium", "low"]` means the LM cannot return "critical"—it will retry. This is compile-time checking for LM behavior.



Where DSPy Genuinely Struggles

Cold Start

You need 20–200 labeled examples before optimization begins. For truly novel tasks, you may have none.

Optimization Cost

MIPROv2 on a 3-module pipeline with 100 examples can cost \$10–30 and take 45–90 minutes. Budget before you commit.

Unfamiliar Mental Model

"Programming with LMs" requires thinking like a compiler engineer. Teams with no ML background often resist this shift.

Poor Fit for Exploration

If you can't define a metric because you don't know what "good" looks like yet, DSPy cannot help. AutoGen or CrewAI serve that space better.

Retrieval Integration Seam

DSPy's Retrieve abstraction works, but integrating Pinecone, Weaviate, or hybrid BM25+dense requires more plumbing than standalone RAG libraries.

Know the Landscape: How DSPy Fits

📌 This lecture focuses on DSPy. The other modules cover CrewAI and AutoGen. This table gives you enough context to choose.

Dimension	DSPy	CrewAI	AutoGen
Core abstraction	Compiled LM program	Role-based agent team	Multi-agent conversation
Prompt optimization	☑ Automatic (MIPROv2, GEPA)	✗ Manual	✗ Manual
Coordination style	Pipeline (typed I/O)	Task handoff	Message passing
Code execution	ProgramOfThought module	Via custom tool	First-class (CodeExecutorAgent)
Best fit	Repeatable typed tasks with metrics	Structured role-based workflows	Exploratory and code-heavy tasks
Worst fit	Exploratory/conversational	Open-ended/adaptive	Auditability-critical pipelines
Key cost risk	Optimization run budget	Multi-agent LM cost per kickoff	Token accumulation per conversation

📌 **Combine when:** Use DSPy as the "brain" inside a CrewAI agent or AutoGen assistant. Use DSPy modules to give agents optimized sub-skills.

The Four Questions Before You Commit to DSPy

Can you define "correct output" well enough to write a metric function?

☒ Use DSPy When

- The task is repeatable: same kind of input → same kind of output, thousands of times.
- You can collect 20–200 labeled examples with correct outputs.
- Output type and structure are fixed: classification, extraction, RAG Q&A, summarization.
- You need model-agnostic pipelines that survive backend changes.
- Reproducibility and auditability are non-negotiable (regulated, compliance-sensitive).

☐ Don't Use DSPy When

- The task is exploratory—you don't know what "good" looks like yet (start with AutoGen).
- You need agents responding to each other dynamically in natural language.
- You need role-based workflow handoffs with structured task outputs (use CrewAI).
- You have zero labeled examples and cannot define a scoring function.
- The team needs a no-code interface (consider AutoGen Studio or CrewAI enterprise tooling).

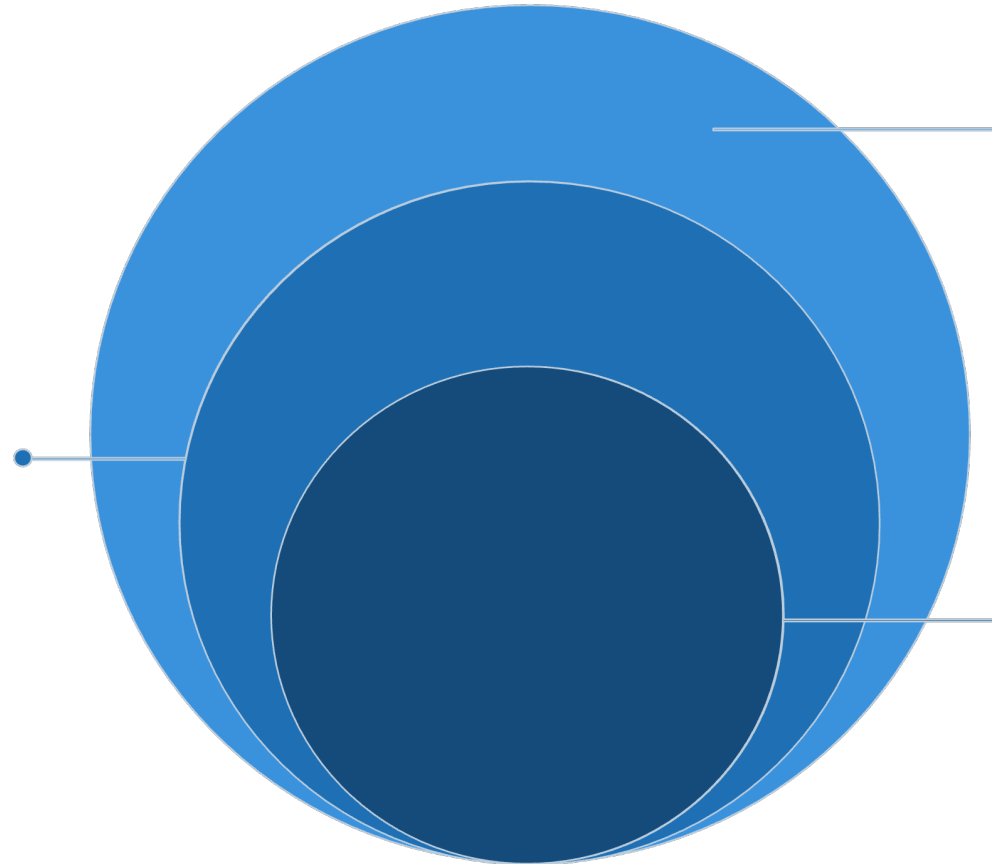
02

Signatures: The Language of Tasks

DSPy: Three Interlocking Layers

Modules

ChainOfThought, ReAct,
ProgramOfThought defines
HOW



Optimizers

BootstrapFewShot, MIPROv2,
GEPA auto-tune prompts and
demos

Signatures

Typed I/O fields + behavioral
contract defines WHAT

Three layers, each independently configurable. Swap the optimizer without touching the Signature. Swap the Module without rewriting the optimization. This separation of concerns is the whole point of the architecture.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Design Pattern 1: The Typed Contract

A Signature is a contract between your pipeline and the LM. Every field is a constraint, every type annotation is enforcement, and the docstring is the behavioral specification. Write it like an API spec—not like a prompt.

```
from typing import Literal
import dspy

#
```

Scaffold 1: Write Your First Typed Contract

📄 *Scenario: Customer support ticket classifier. Your team currently maintains a 200-line .txt prompt file. You're converting it to DSPy.*

01

Write the Signature

Define `ExtractTicketInfo(dspy.Signature)`.
Inputs: `ticket_text (str)`. Outputs: `category (Literal['billing', 'technical', 'general'])`,
`urgency, summary (str), requires_human (bool)`.

02

Add the Behavioral Rule

Add docstring rule: *"If in doubt, set urgency='high' and requires_human=True. Never leave a billing issue marked general."* This rule lives in the docstring, not in code logic.

03

Instantiate and Run

Instantiate `dspy.Predict(ExtractTicketInfo)`. Call it on three real tickets from your domain. Print the full result object.

04

Change and Observe

Edit one docstring rule and re-run. Observe how behavior changes without touching any field or code outside the Signature.

05

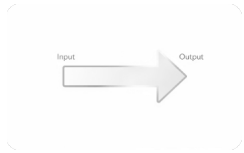
Conceptual Test

A colleague asks: "How does the LM know the rules?" Walk them through exactly where the behavioral contract lives and how the optimizer uses it.

03

Modules: Composable Reasoning Patterns

Choose the Right Reasoning Pattern First



Predict

Direct I/O. No reasoning step.

Use for classification, extraction, and structured transforms.

Fastest and cheapest — start here and upgrade only if accuracy is insufficient.



ChainOfThought

Adds a reasoning scratchpad before the answer. Research shows **15–40% accuracy gains** on multi-step tasks (Wei et al., NeurIPS 2022) at 1.5–2× the token cost. Use as default when Predict falls short.



ReAct

Reason + Act loop with interleaved tool calls. Use when the task requires external information at inference time: web search, database queries, calculators. Costs **3–5× Predict**.



ProgramOfThought

Writes executable Python and runs it—best for numerical tasks where reasoning in natural language is unreliable. Ideal for computation-heavy pipelines.

Design Pattern 2: Pipeline Composition

```
import dspy

class AnswerWithCitations(dspy.Signature):
    """Answer based solely on retrieved context.
    Use [1], [2] citation markers for every claim."""
    question: str = dspy.InputField()
    context: list[str] = dspy.InputField(
        desc="Retrieved passages, indexed from 1")
    answer: str = dspy.OutputField(
        desc="Answer with [N] citation markers")
    citations: list[int] = dspy.OutputField(
        desc="Indices of passages cited")

class MultiHopRAG(dspy.Module):
    """Two-hop retrieval: refine query, then answer."""
    def __init__(self, num_passages=4):
        self.refine_query = dspy.ChainOfThought(
            "context, question -> search_query"
        )
        self.generate = dspy.ChainOfThought(
            AnswerWithCitations())

    def forward(self, question: str, retriever_fn):
        passages = []
        query = question
        # Hop 1: retrieve context, refine query
        ctx = retriever_fn(query)
        passages.extend(ctx)
        query = self.refine_query(
            context=ctx, question=question
        ).search_query
        # Hop 2: retrieve + generate answer
        passages.extend(retriever_fn(query))
        return self.generate(
            question=question, context=passages)
```

A DSPy Module is a Python class where each sub-module is a named attribute and `forward()` defines the data flow. Composition over monoliths—small, testable modules that chain together.

Composition Rules

Each sub-module (`self.refine_query`, `self.generate`) is independently optimized in one pass.

`forward()` is pure Python — use any control flow: loops, conditionals, branching.

Inject `retriever_fn` as a parameter to keep the module retrieval-backend agnostic.

The optimizer tunes each sub-module's prompts simultaneously in one optimization call.

 **Pattern check:** If your `forward()` exceeds 20 lines, consider splitting into two Modules.

Do NOT inject `retriever_fn` as a `forward()` argument in production. `** dspy.asyncify` cannot handle non-dspy callable arguments, and the optimizer cannot trace injected callables. Instead, declare `self.retrieve = dspy.Retrieve(k=4)` in `__init__` and call it inside `forward()`. The pattern here is for teaching composition—not for production deployment.

Scaffold 2: Build and Introspect a Pipeline

❏ *Scenario: Article summarization → quiz generation. A two-stage pipeline where the second stage depends on the first.*

01

Write Summarize

Define `Summarize(dspy.Signature): article (str) → summary (str, desc="max 100 words")`. Wrap in `dspy.ChainOfThought`.

02

Write GenerateQuiz

Define `GenerateQuiz(dspy.Signature): summary (str) → questions (list[str], desc="exactly 3 distinct questions")`. Wrap in `dspy.Predict`.

03

Compose StudyHelper

Build `StudyHelper(dspy.Module)`. In `forward()`: call `summarize`, then pass the summary to `generate_quiz`.

04

Run and Verify

Run on a Wikipedia article. Verify: (a) summary is under 100 words, (b) exactly 3 questions are generated.

05

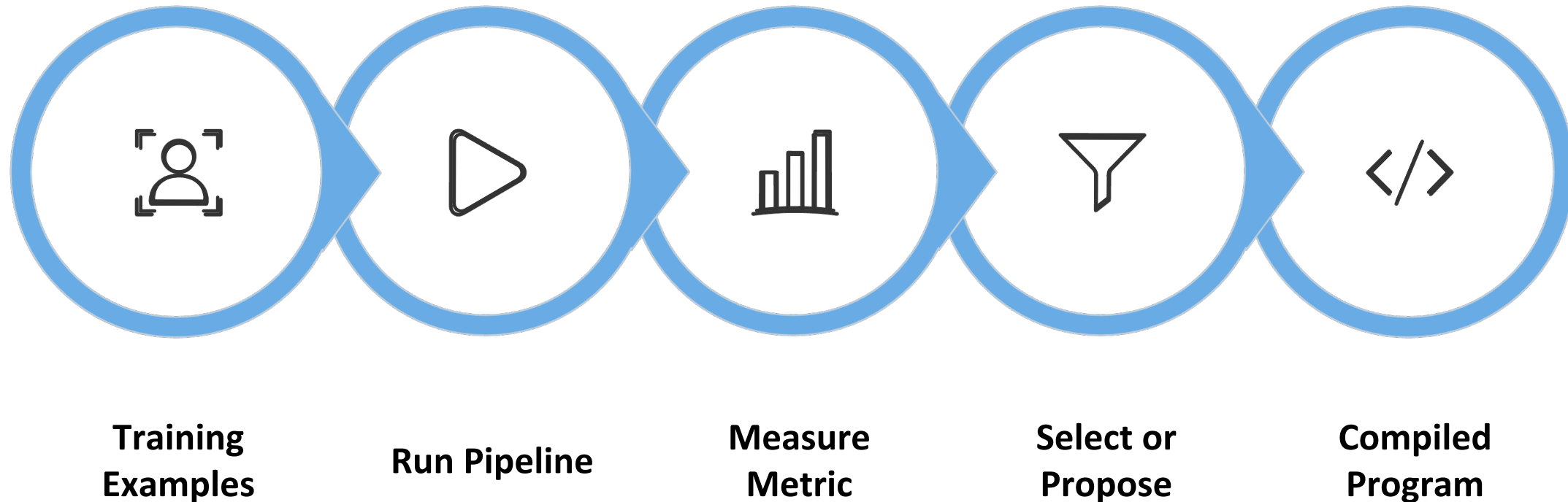
Introspect the Prompts

Call `dspy.inspect_history(n=2)`. Where did your `desc=` appear? Where did the docstring appear? This is exactly what the optimizer will work with.

04

Optimizers: Auto-Tuning Your Pipeline

How Optimization Works: The 5-Step Loop



BootstrapFewShot runs this loop in ~2 minutes. **MIPROv2** runs many candidates via Bayesian search—30–90 minutes. **GEPA** (DSPy 3.x) uses reflective prompt evolution and has been shown to match or exceed RL-based approaches on several benchmarks (Agrawal et al., arXiv:2507.19457, 2025).

Design Pattern 3: Metric-First Development

```
import dspy
from dspy.evaluate import Evaluate

def answer_quality(example, pred, trace=None) -> float:
    """Composite: 60% accuracy + 40% conciseness."""
    gold = set(example.answer.lower().split())
    pred_ = set(pred.answer.lower().split())
    if not pred_: return 0.0
    p = len(gold & pred_) / len(pred_)
    r = len(gold & pred_) / len(gold)
    f1 = 2 * p * r / (p + r + 1e-9)
    word_count = len(pred.answer.split())
    conciseness = 1.0 if word_count <= 30 else \
        max(0, 1 - (word_count - 30) / 50)
    return 0.6 * f1 + 0.4 * conciseness

# Establish baseline BEFORE any optimization
lm = dspy.LM("openai/gpt-4o-mini", temperature=0.0)
dspy.configure(lm=lm)
baseline = MultiHopRAG()
evaluator = Evaluate(devset=devset,
    metric=answer_quality, num_threads=8)
baseline_score = evaluator(baseline)
print(f"Baseline: {baseline_score:.3f}") # e.g. 0.512

# Optimize – compare to baseline, not intuition
from dspy.teleprompt import BootstrapFewShot
optimizer = BootstrapFewShot(
    metric=answer_quality,
    max_bootstrapped_demos=4,
    max_labeled_demos=8)
compiled = optimizer.compile(
    MultiHopRAG(), trainset=trainset)
optimized_score = evaluator(compiled)
print(f"Optimized: {optimized_score:.3f}") # e.g. 0.671

# Only save if there is genuine improvement
if optimized_score > baseline_score + 0.02:
    compiled.save("rag_optimized_v1.json")
    print(f"Gain: +{optimized_score - baseline_score:.3f}")
```

Write your metric function before you write your pipeline. If you cannot define what "correct" means as a numeric score, DSPy cannot help. The metric IS your quality specification—treat it with the same rigor as production code.

Metric-First Rules

Write the metric before the pipeline — this forces you to define "done."

A metric that always returns 1.0 is a bug, not a feature.

Always measure baseline first — you cannot claim improvement without a starting point.

Composite metrics (accuracy + conciseness) encode real engineering trade-offs.

If optimized > baseline + 0.02, the gain is real — less than that is noise.

 **The metric is the real specification.** A vague metric produces a vague, unreliable program.

Choosing the Right Optimizer: A Decision Guide

Optimizer	How It Works	When to Use	Data Needed	Cost
BootstrapFewShot	Collects successful traces as few-shot demos	Quick first win; limited budget	20–50 examples	Low (~\$1–3)
MIPROv2	Bayesian search over instructions + demo sets jointly	Maximum accuracy; budget available	100–200 examples	Medium–High (\$10–30)
GEPA (3.x)	Reflective prompt evolution; iterative self-improvement	When MIPROv2 plateaus; complex multi-step tasks	50+ examples	Medium–High
BootstrapFinetune	Generates training data to finetune LM weights	Small model + highest accuracy at inference	200+ examples	High (finetune)
LabeledFewShot	Inserts your k labeled examples directly—no search	No optimization budget; fast baseline	k examples	Negligible

GEPA citation: Agrawal et al., "GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning," arXiv:2507.19457, Jul 2025. CoT accuracy: Wei et al., NeurIPS 2022.

❏ **Start with BootstrapFewShot.** Upgrade to MIPROv2 if the gain is insufficient. Finetune only if you need sub-10ms inference on a small model.

Section 04—Optimization Principles

Key Takeaways

- Measure before and after every run—a 2% gain is noise, a 15% gain is signal worth keeping.
- Your metric function defines quality—it deserves the same review as production code.
- Save compiled programs as versioned JSON: treat them like model checkpoints.

⚠ Watch-Outs

- **Never share training and evaluation examples**—the optimizer will overfit and the dev score will lie to you.
- MIPROv2 costs real money: budget \$10–30 per full optimization run on a 3-module pipeline.
- If optimization doesn't help, check `.with_inputs()`, metric logic, and example quality—in that order.

05

Assertions: Runtime Correctness Enforcement



Design Pattern 4: Constrained Output With Self-Correction

```
import dspy, json

class SafeExtractor(dspy.Module):
    """Extract entities and sentiment with hard constraints."""
    def __init__(self):
        self.extract = dspy.ChainOfThought(
            "document -> entities_json, sentiment, word_count")

    def forward(self, document: str) -> dspy.Prediction:
        pred = self.extract(document=document)

        # Constraint 1: Structural validity
        try:
            entities = json.loads(pred.entities_json)
            dspy.Assert(isinstance(entities, list),
                f"entities_json must be a JSON array. "
                f"Got type: {type(entities).__name__}")
        except json.JSONDecodeError as e:
            dspy.Assert(False,
                f"entities_json is not valid JSON. "
                f"Error: {e}. Value: {pred.entities_json[:100]}")

        # Constraint 2: Enum enforcement
        valid = ("positive", "negative", "neutral", "mixed")
        dspy.Assert(pred.sentiment in valid,
            f"sentiment must be one of {valid}. "
            f"Got: '{pred.sentiment}'")

        # Constraint 3: Numeric range
        try:
            wc = int(pred.word_count)
            dspy.Assert(1 <= wc <= 9999,
                f"word_count must be 1-9999. Got: {wc}")
        except ValueError:
            dspy.Assert(False,
                f"word_count must be integer. "
                f"Got: '{pred.word_count}'")

        # Suggestion (soft): non-blocking nudge
        dspy.Suggest(len(entities) <= 10,
            "Prefer extracting the top 10 most important entities")
        return pred
```

Assertions enforce structural correctness at runtime and inject failure feedback into the LM's next attempt. The LM sees your error message and retries. Design assertions as error messages to a junior developer—specific, actionable, and diagnostic.

Assertion Design Rules

`Assert(False, msg)` injects `msg` as feedback and retries — the LM reads your error message directly.

Always include the actual value in the message: "Got 'urgent', expected one of [...]"

`dspy.Suggest` = soft nudge (logs warning, does not block). Use for preferences, not correctness.

Fail rate >15% on a constraint = redesign the Signature's output field, not the Assert.

Assertions are active during optimization — they filter out bad training demos automatically.

One Assert per constraint. Combining two conditions produces useless error messages on retry.

Scaffold 3: Constrain Your Classifier

- ❑ *Scenario: Harden the ticket classifier from Exercise 1 with runtime constraints that act as a safety net when the LM goes off-script.*

01

Assert Category

Add `dspy.Assert` that category is one of `['billing', 'technical', 'general']`. Include the actual value returned in the error message.

02

Assert Urgency

Add `dspy.Assert` that urgency is one of `['high', 'medium', 'low']`.

03

Assert Summary Length

Add `dspy.Assert(len(pred.summary.split()) <= 25, ...)`. Include the actual word count in the error message.

04

Test Adversarially

Run with: `"LEGAL ACTION WILL BE TAKEN IMMEDIATELY IF NOT RESOLVED."` Verify `requires_human=True` and `urgency='high'`.

05

Find the Retry Message

Call `dspy.inspect_history()` after a failure. Find the injected feedback. What did the LM see? Why did it correct itself? This is the self-correction mechanism made visible.

06

Evaluation and Production Deployment

Production Design: Treat Compiled Programs Like Model Artifacts

Load Compiled JSON Once at Startup

Loading per-request re-reads disk on every call.
Load in `__init__`, not in the request handler.
Your optimization is never applied otherwise.

Encapsulate retrieval inside the module — never inject it as a
`'forward()'` argument.

Health Check = Inference Check

`GET /health` should call `async_program("ping")` and return 503 if it fails. A process that starts but can't inference is not healthy.

Version Like Model Checkpoints

`rag_v1.json` → `rag_v2.json` when you re-optimize.
Keep both. Roll back if production metrics drop.

Instrument With OpenTelemetry

DSPy integrates with Arize Phoenix and MLflow via `openinference-instrumentation-dspy`. This gives per-request prompt traces, latency histograms, and metric drift alerts.

Re-Optimize on a Schedule

Data distributions shift. A program compiled on winter queries will drift on summer traffic. Monthly re-optimization is a reasonable starting cadence.

Test Before You Ship

Unit test Signatures with known inputs and assert output types. Integration test with `dspy`. Evaluate on a held-out slice. A passing eval is your deploy gate.



FastAPI Service—Production Pattern (DSPy 3.x)

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import dspy, time, logging

app = FastAPI(title="DSPy QA Service")
logger = logging.getLogger(__name__)

# AnswerWithConfidence Signature (defined earlier in the deck):
# question: str, context: str → answer: str, confidence:
# Literal["high", "medium", "low"]
#
# CompiledQA wraps it with self-contained retrieval so the API surface
# is simply: question in → answer + confidence out.
# dspy.asyncify works correctly because forward() has no injected callables.
class CompiledQA(dspy.Module):
    def __init__(self):
        self.predict = dspy.ChainOfThought(AnswerWithConfidence)

    def forward(self, question: str) -> dspy.Prediction:
        context = retrieve_context(question) # your retrieval logic here
        return self.predict(question=question, context=context)

class AppState:
    """Load program once at startup – not per request."""
    def __init__(self):
        lm = dspy.LM("openai/gpt-4o-mini",
                     temperature=0.0, cache=True)
        dspy.configure(lm=lm)
        self.program = CompiledQA()
        self.program.load("qa_optimized_v1.json") # versioned checkpoint
        self.async_program = dspy.asyncify(self.program)

state = AppState()

class QueryRequest(BaseModel):
    question: str

class QueryResponse(BaseModel):
    answer: str
    confidence: str # matches AnswerWithConfidence.confidence field
    latency_ms: float

@app.post("/query", response_model=QueryResponse)
async def query(req: QueryRequest):
    if not req.question.strip():
        raise HTTPException(400, "Question cannot be empty")
    t0 = time.perf_counter()
    try:
        result = await state.async_program(question=req.question)
    except Exception as e:
        logger.error(f"Inference error: {e}")
        # litellm.RateLimitError (429) → retry with backoff, not 500
        # litellm.AuthenticationError → alert on-call, don't retry
        raise HTTPException(500, str(e))
    ms = (time.perf_counter() - t0) * 1000
    logger.info(f"q={req.question[:40]!r} latency={ms:.1f}ms")
    return QueryResponse(answer=result.answer,
                        confidence=result.confidence,
                        latency_ms=round(ms, 1))

@app.get("/health")
async def health():
    try:
        await state.async_program(question="health check ping")
        return {"status": "ok", "model": "gpt-4o-mini"}
    except Exception as e:
        raise HTTPException(503, f"Inference unavailable: {e}")

```

Take-Home Lab: Full Optimization + Deploy Workflow

❑ *Budget 2–3 hours with API access. Beyond the scope of a short workshop.*

❑ *Scenario: End-to-end on HotPotQA—define metric, establish baseline, optimize, compare, deploy.*

01

Load and Split HotPotQA

Split: 100 train, 50 dev, 50 test. **Never touch test until the very end.** Evaluate baseline MultiHopRAG on dev. Record your baseline score.

02

Write a Composite Metric

60% exact_match + 40% conciseness (penalty for answers over 30 words). Justify the 60/40 split in one sentence—this is a real product decision.

03

Run BootstrapFewShot

Compare dev score to baseline. Open compiled.json—read which demos were selected. Why these and not others?

04

Run MIPROv2

Use auto='light'. Compare to BootstrapFewShot. What did instruction search find that demo selection alone couldn't?

05

Evaluate on the Test Set (Once)

Run your best compiled program on the test set exactly once. Does it match dev scores? Discuss what overfitting to dev looks like in practice.

07

Debugging and Failure Mode Taxonomy

Four Failure Modes and Their Signatures



Structural Failure

Symptom: Optimizer runs but score doesn't improve (<2%). **Causes:** Missing `.with_inputs()`; metric bug; mislabeled examples. **Fix:** `print(example.inputs())`; unit-test metric by hand on two examples.



Overfitting Failure

Symptom: Dev score high but test score is 10+ points lower. **Cause:** Training and dev share overlapping examples. **Fix:** Strict train/dev/test split. The optimizer must never see the test set.



Assert Death Spiral

Symptom: Every request retries 3 times and fails. Assert failure rate >50%. **Cause:** The LM doesn't understand the required format. **Fix:** Add `format_hint` to the output field's `desc=`.



Drift Failure

Symptom: Production accuracy degrades over 4–6 weeks with no code changes. **Cause:** Input distribution shifted; compiled demos no longer represent real queries. **Fix:** Collect recent prod examples, re-evaluate, re-optimize if score drops >5%.

Debugging Toolkit (DSPy 3.x)

```
import dspy, json

# Tool 1: What did DSPy actually send to the LM?
# The most reliable debugging tool in DSPy 3.x.
dspy.inspect_history(n=3)
# Prints: full prompt text, raw model response, parsed fields.
# Compare what you expected vs. what it actually says.

# Tool 2: Trace a single example through the full pipeline.
# dspy.context(trace=[]) captures traces in the context object.
with dspy.context(trace=[]) as ctx:
    result = program(question="What caused the 2008 financial crisis?")
# inspect_history shows each module call in order:
# inputs, outputs, and reasoning steps.
dspy.inspect_history(n=5)

# Tool 3: Diagnose Assert failures.
# After a failure, inspect_history and look for messages
# containing "Your response failed:" – that IS what the LM
# saw on the retry turn. If your error message is vague,
# the retry will also be vague.
dspy.inspect_history(n=10)

# Tool 4: Read what the optimizer actually changed.
data = json.load(open("compiled.json"))
for key, val in data.items():
    if "demos" in key.lower() or "instructions" in key.lower():
        print(f"\n[{key}]\n")
        print(json.dumps(val, indent=2)[:500])
# This is the ground truth of what optimization produced.
# If the instructions look wrong, your metric or docstring is wrong.
```

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited

What You'll Still Know When DSPy Is Deprecated

1

Separate Structure from Parameters

In neural networks, architecture is separate from weights. In LM pipelines, program structure should be separate from prompt text. Any future framework that blurs this line will be harder to maintain and debug.

2

Define the Metric Before the Pipeline

You cannot optimize toward a goal you haven't defined. If you cannot write a scoring function for your task, you don't understand it well enough to automate it. This is true in ML, product, and management.

3

Measure Everything, Trust Nothing

Intuition about prompt quality is unreliable. A prompt that "feels better" needs a number to prove it. Always measure baseline, always hold out test data, and require a minimum delta ($\geq 2-5\%$) before claiming improvement.

4

Typed Contracts Catch Errors Earlier and Cheaper

Every type annotation and every Literal field moves error detection from runtime (when a user sees it) to definition time (when a developer sees it). Invest in types: this principle applies to every system you will ever build.

DSPy—Three Principles to Carry Forward

Key Takeaways



Measure Before You Optimize

Define your metric and collect 20–50 examples before choosing any optimizer.



Constraints Are Features

Type annotations, literal fields, and assertions make LM behavior predictable at scale.



Start Simpler Than You Think

A well-configured ChainOfThought often outperforms a complex multi-module pipeline.

⚠ Watch-Outs



Re-Optimize Monthly in Production: Your data distribution shifts, and static compiled programs drift.



DSPy Is a Tool, Not a Solution: A poorly designed Signature optimizes reliably toward the wrong goal.



Pin Your Version: `dspy==3.1.3` in `requirements.txt`—the API evolves quickly and breaking changes occur between minor versions.

DSPy 3.x Quick Reference

Concept	Key Insight	Use When	Notes
Signatures	Typed I/O + docstring behavioral contract	Every LM call	Write like an API spec
Predict	Direct I/O, no reasoning step	Classification, extraction	Fastest and cheapest—start here
ChainOfThought	Reasoning scratchpad; +15–40% accuracy on reasoning tasks [†]	Multi-step, inference tasks	Default upgrade from Predict
ReAct	Reason + Act with tool calls	Needs external data at inference	3–5× cost of Predict
ProgramOfThought	Writes + executes Python	Numerical, computational tasks	Unreliable reasoning in NL avoided
BootstrapFewShot	Traces → demos; fast	20+ examples, limited budget	~\$1–3, ~2 min
MIPROv2	Bayesian search: instructions + demos	100+ examples, max accuracy	\$10–30, 30–90 min
GEPA (3.x)	Reflective prompt evolution	MIPROv2 plateau, complex tasks	arXiv:2507.19457
Assertions	Hard constraint + auto-retry with feedback	Output correctness required	One Assert per constraint
dspy.Suggest	Soft nudge, non-blocking	Style preferences	Logs warning, doesn't block
asyncify()	Async wrapper, zero code change	FastAPI, high-throughput serving	Load program once at startup
compiled.save()	JSON serialization of optimized program	Always	Version like a model checkpoint

[†] Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," NeurIPS 2022.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited